

Top Strategies to Reduce Latency



BYTEBYTEGO

JAN 23, 2025 • PAID



317



2



15

Share

Latency is a fundamental concept to consider when designing any application. It refers to the delay between a user's action and the system's response.

In technical terms, latency measures the time it takes for data to travel from a source to a destination and back. It is usually expressed in milliseconds (ms) and is critical in determining the speed and responsiveness of applications, websites, and networks.

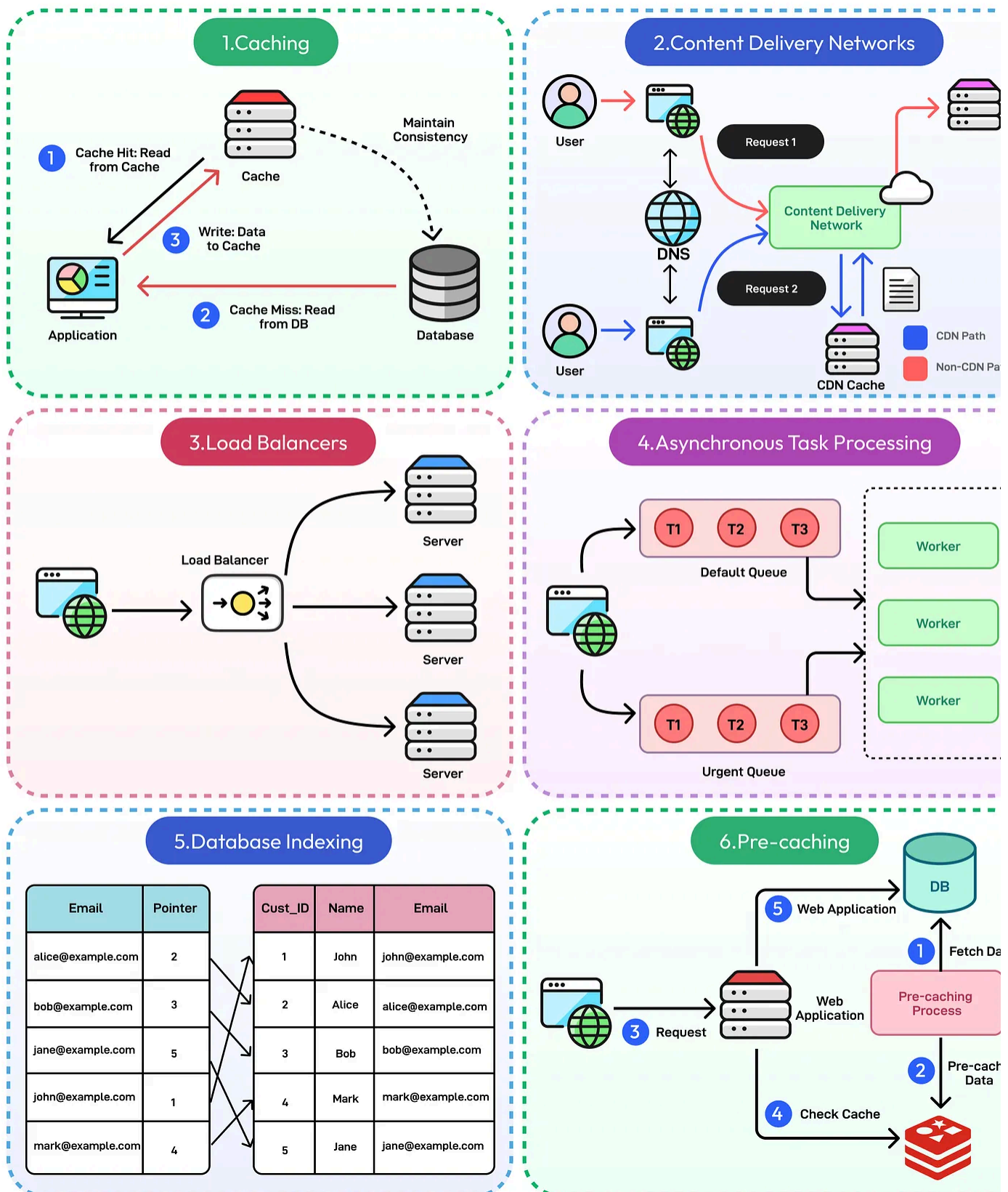
High latency can have a profound impact across business domains:

- **User Experience:** Slow-loading websites frustrate users and lead to higher bounce rates. A Google study shows that slowing down the search results page by 100 to 400 milliseconds has a measurable impact on the number of searches per user, ranging from -0.2% to -0.6%.
- **Business Operations:** Delays in real-time applications like video conferencing and online collaboration tools can hamper productivity. Due to delayed page loads, commerce platforms suffer from reduced customer satisfaction and lower sales. For example, Amazon estimates that a 1-second increase in latency could cost \$1 billion annually in sales.
- **SEO Rankings:** Search engines prioritize fast-loading websites in search rankings. For example, Google's Core Web Vitals places significant weight on metrics like First Input Delay (FID), which is directly tied to latency. High latency increases page load times, negatively affecting SEO and organic traffic.

In this article, we will learn about latency and its types in detail. We will explore top strategies to reduce latency, such as caching, content delivery networks (CDNs), load

balancing, asynchronous processing, database indexing, pre-caching, data compression, and connection reuse.

Top Strategies to Reduce Latency



Latency vs Bandwidth vs Throughput

Before we examine the strategies for low latency, let us clarify a few terminologies closely associated with latency, namely bandwidth and throughput.

- **Latency:** As mentioned earlier, latency is the time delay in transferring data. For example, when we click on a website link, latency is the time it takes for the request to reach the server and for the server's response to return.
- **Bandwidth:** The maximum amount of data that can be transmitted over a network in a given period. For example, a 100 Mbps connection means up to 100 megabits of data can be transmitted per second.
- **Throughput:** The amount of data transferred over the network in a specific time, factoring in real-world conditions.

To summarize, bandwidth is about capacity, throughput measures usage, and latency defines delay. Even with high bandwidth, high latency can lead to slow performance.

Types of Latency

Latency can arise from multiple stages in the data processing and delivery pipeline. Each type of latency stems from different causes and impacts system performance in a specific way.

1 - Network Latency

Network latency refers to the time it takes for data to travel between the client (for example, a browser or device) and the server across a network. The farther the data must travel, the higher the latency.

The primary causes of this type of latency are as follows:

- **Physical Distance:** The farther the data must travel the higher the latency. For example, a user in New York accessing a website hosted in Singapore may experience high network latency due to long physical distance.
- **Routing:** Packets may take multiple hops between intermediary routers, increasing travel time.
- **Congestion:** Network traffic overload can slow down data transfer.
- **Protocol Overhead:** Network protocols like TCP/IP introduce additional delays especially during handshake processes.

2 - Server Latency

Server latency refers to the time it takes for a server to process a request and send a response back to the client.

The primary causes are as follows:

- **Resource Constraints:** Overloaded servers may take longer to handle requests.
- **Inefficient Code:** Poorly written server-side code can delay request handling.
- **Database Performance:** Complex or unoptimized database queries increase processing time. Also, searching large datasets without proper indexing results in delays.
- **Backend Architecture:** Ineffective architecture, such as using a single-threaded server for heavy workloads, can bottleneck performance.

3 - Client-Side Latency

Client-side latency refers to delays caused on the user's device (for example, browser app) while rendering or processing received data.

The primary causes are as follows:

- **Rendering Delay:** This can be caused by large and unoptimized images, videos, CSS/JavaScript files. Also, inefficient rendering engines struggle with complex layouts or animations.
- **Inefficient Code Execution:** Blocking JavaScript operations that freeze the main thread and not using lazy loading for non-critical resources can increase the latency.
- **Device Limitations:** Older devices with lower specs may take longer to process data.

Strategies to Reduce Latency

Let us look at some of the most popular strategies for reducing latency.

These strategies are mentioned in no particular order. Also, we don't need to use all them in every project. The applicability of a strategy depends on the circumstances the project.

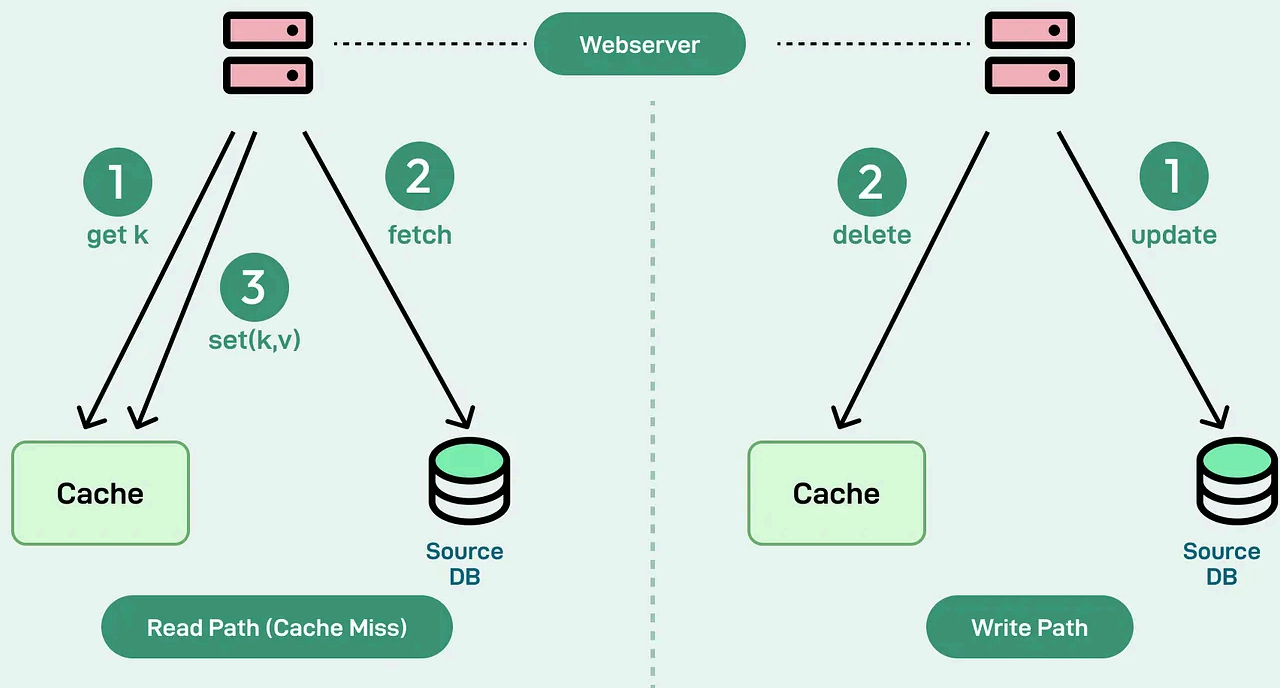
1 - Caching

Caching is a technique for temporarily storing data in a temporary location known the cache, to make it faster to access.

When a client requests data, the system first checks the cache. The cache is faster-access storage such as RAM, disk, or even edge servers. If the data is found in the cache (a "cache hit"), it is returned directly. If not (a "cache miss"), it is fetched from the source and added to the cache for future use.

See the diagram below that shows how a basic caching approach can work for the read path and the write path.

How Caching Works



Caching minimizes latency and improves performance by reducing the need to fetch data from its source repeatedly. It is particularly effective in scenarios involving frequently accessed, unchanging data.

Some best practices for effective caching are as follows:

- **Define Appropriate TTL (Time-to-Live) Values:** Set an expiration time for cache items to ensure outdated data is replaced.
- **Invalidate Stale Data:** Implement strategies to clear or update outdated cache entries. For example, clear specific keys when updates occur. Also, use versioned keys to avoid serving old data.
- **Cache Only Frequently Accessed Data:** Identify and cache data that is requested often or is expensive to compute. Avoid caching data that changes frequently unless a robust invalidation mechanism is in place.

- **Optimize Cache Size:** Ensure that the cache has enough capacity to store frequently accessed data but is not so large that it impacts performance.
- **Use Layered Caching:** Combine multiple caching strategies for optimal performance. For example, caching at the browser level and also, on the server side.
- **Analyze Cache Performance:** Monitor cache hit and cache miss ratios to fine-tune cache size. Also, check cache usage and response times.

2 - Content Delivery Networks (CDNs)

A Content Delivery Network (CDN) is a network of geographically distributed servers that work together to deliver content (such as images, videos, and web pages) quickly and efficiently to users.

CDNs reduce latency by caching content closer to users, minimizing the time it takes for data to travel from the server to the end user. CDN providers place edge servers at multiple locations, reducing the physical distance data needs to travel.

Here's a step-by-step explanation of how CDNs work:

- **Content Replication:** A CDN replicates and caches static content from the origin server (for example, images, CSS, JavaScript, videos) on multiple edge servers located around the world.
- **User Request:** When a user accesses a website or application, their request is routed to the nearest CDN edge server based on location.
- **CDN Response:** If the requested content is cached on the edge server (a cache hit), it is served directly to the user, reducing the need to contact the origin server. If the content is not cached (a cache miss), the edge server fetches it from the origin server, caches it, and then delivers it to the user.
- **Load Balancing and Optimization:** CDNs dynamically manage traffic to prevent overload on any single server and ensure consistent performance during high-

traffic periods.

See the diagram below that shows how a CDN works.

By handling the majority of user requests, CDNs reduce the load on origin servers, allowing them to focus on dynamic content and backend processes.

Some key benefits of CDNs are as follows:

- Cached assets like images, videos, CSS, and JavaScript are delivered more quickly to users, improving page load speeds.
- CDNs enable websites and apps to serve users globally without significant latency.
- CDNs often include features like traffic filtering and rate limiting to protect against Distributed Denial of Service (DDoS) attacks.

- By caching and serving data from edge servers, CDNs decrease the volume of data fetched from the origin server, reducing bandwidth usage and associated costs.

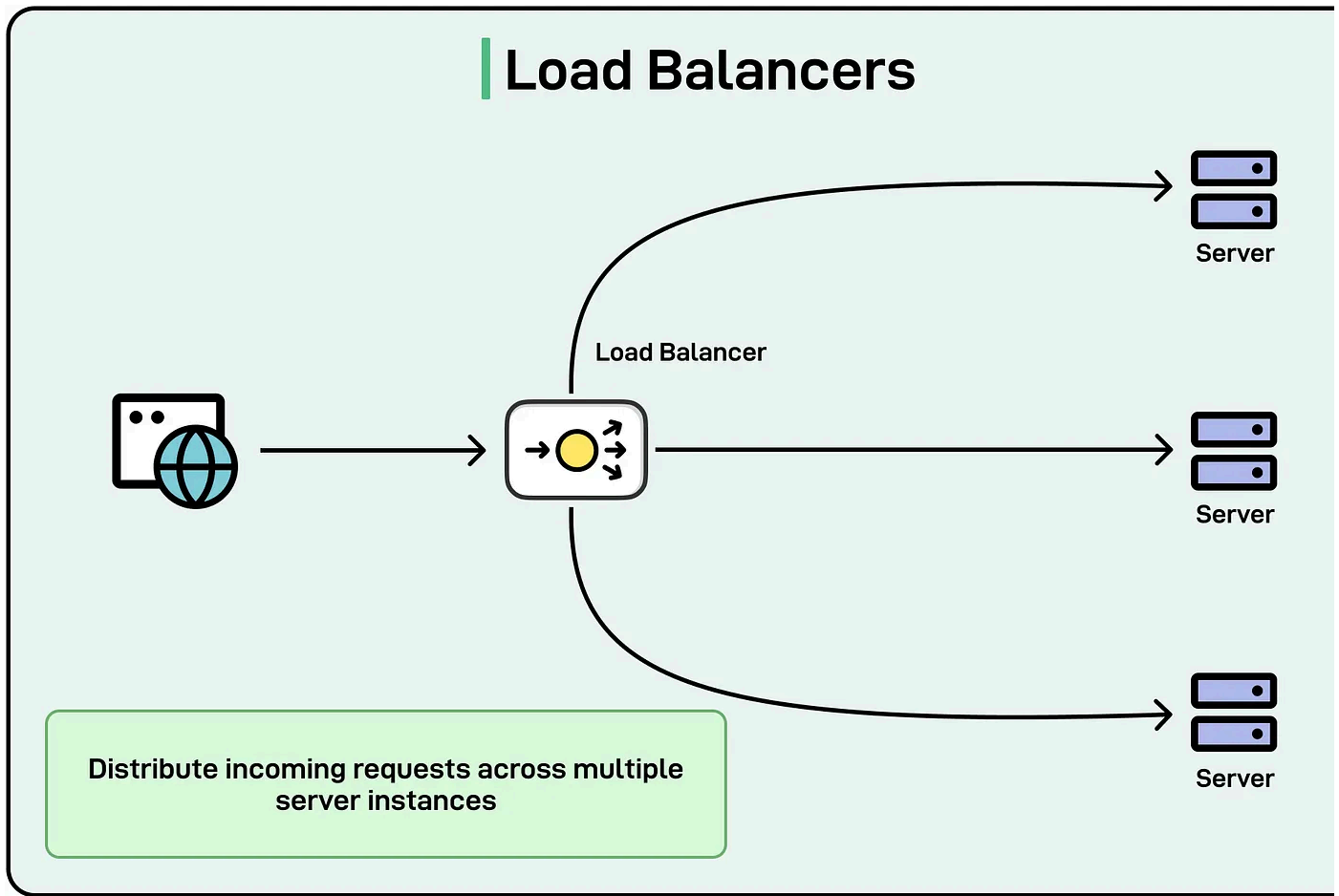
3 - Load Balancing

Load balancing is the process of distributing incoming traffic or workload across multiple servers to ensure no single server is overwhelmed.

Balancing traffic across multiple servers prevents bottlenecks, reducing the time taken to process requests. By rerouting traffic from failed servers to healthy ones, load balancers ensure continuous service availability.

Here's how load balancing works:

- **Traffic Distribution:** A load balancer sits between the client and the server pool, intercepting incoming requests. It evaluates the state of the servers (availability, load, etc.) and directs traffic accordingly.
- **Health Monitoring:** Load balancers periodically check the health of servers to ensure requests are not sent to unresponsive or overloaded servers.

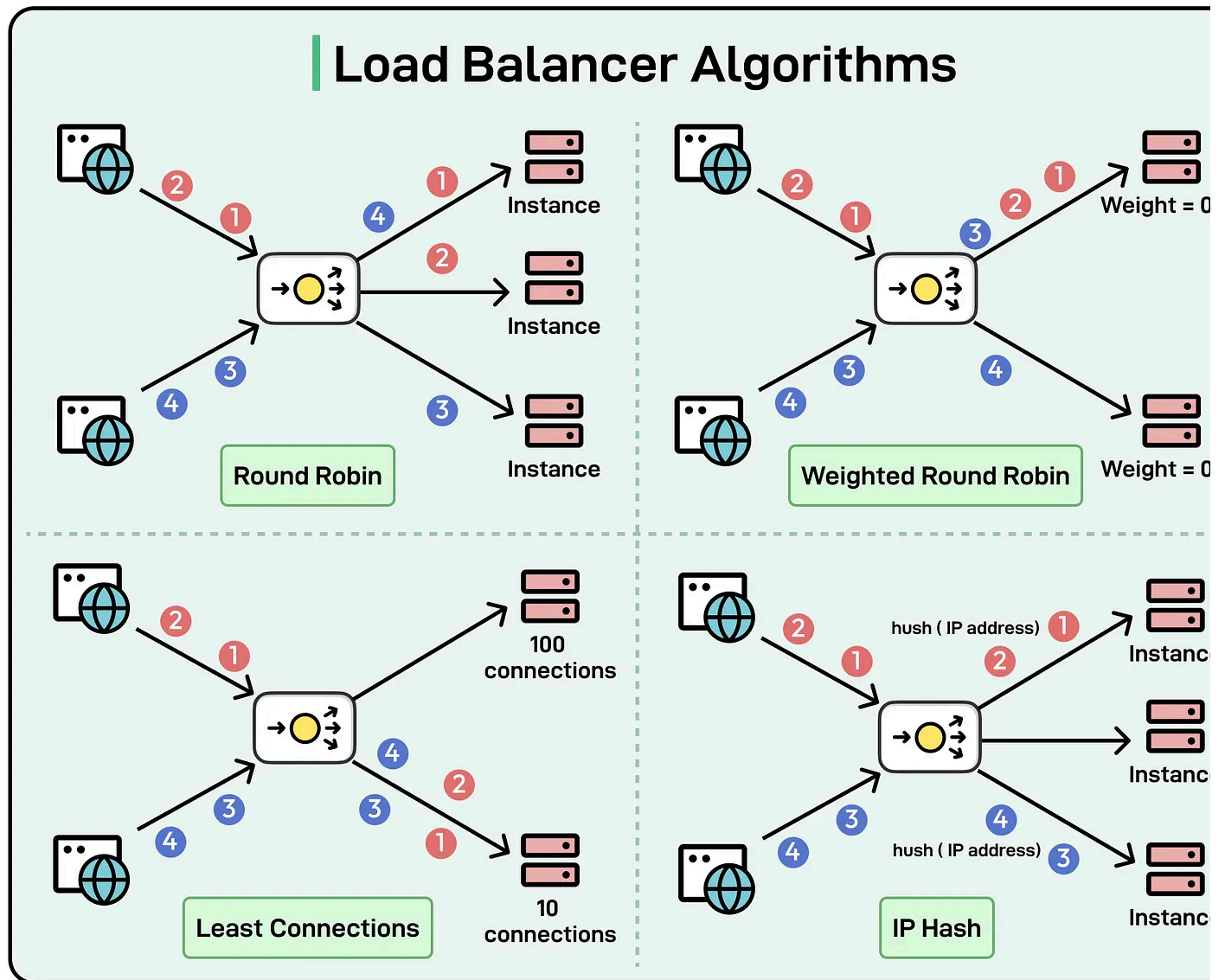


Some key load-balancing algorithms are as follows:

- **Round-Robin:** Requests are distributed sequentially among servers in the pool. It is simple and effective for evenly loaded servers. However, this approach does not account for varying capacities or performance levels.
- **Weighted Round-Robin:** This algorithm assigns weights to servers based on their capacity, distributing more requests to higher-capacity servers. It balances traffic based on server performance.
- **Least Connections:** Traffic is directed to the server with the fewest active connections. It adapts to traffic variations and ensures better resource utilization.
- **IP Hash:** Uses the client's IP address to determine which server will handle the request. This ensures the same client is always routed to the same server.

- **Geographic Load Balancing:** Routes traffic based on the user's geographic location to the nearest server.

See the diagram below that shows the main algorithms in a visual manner



4 - Asynchronous Processing

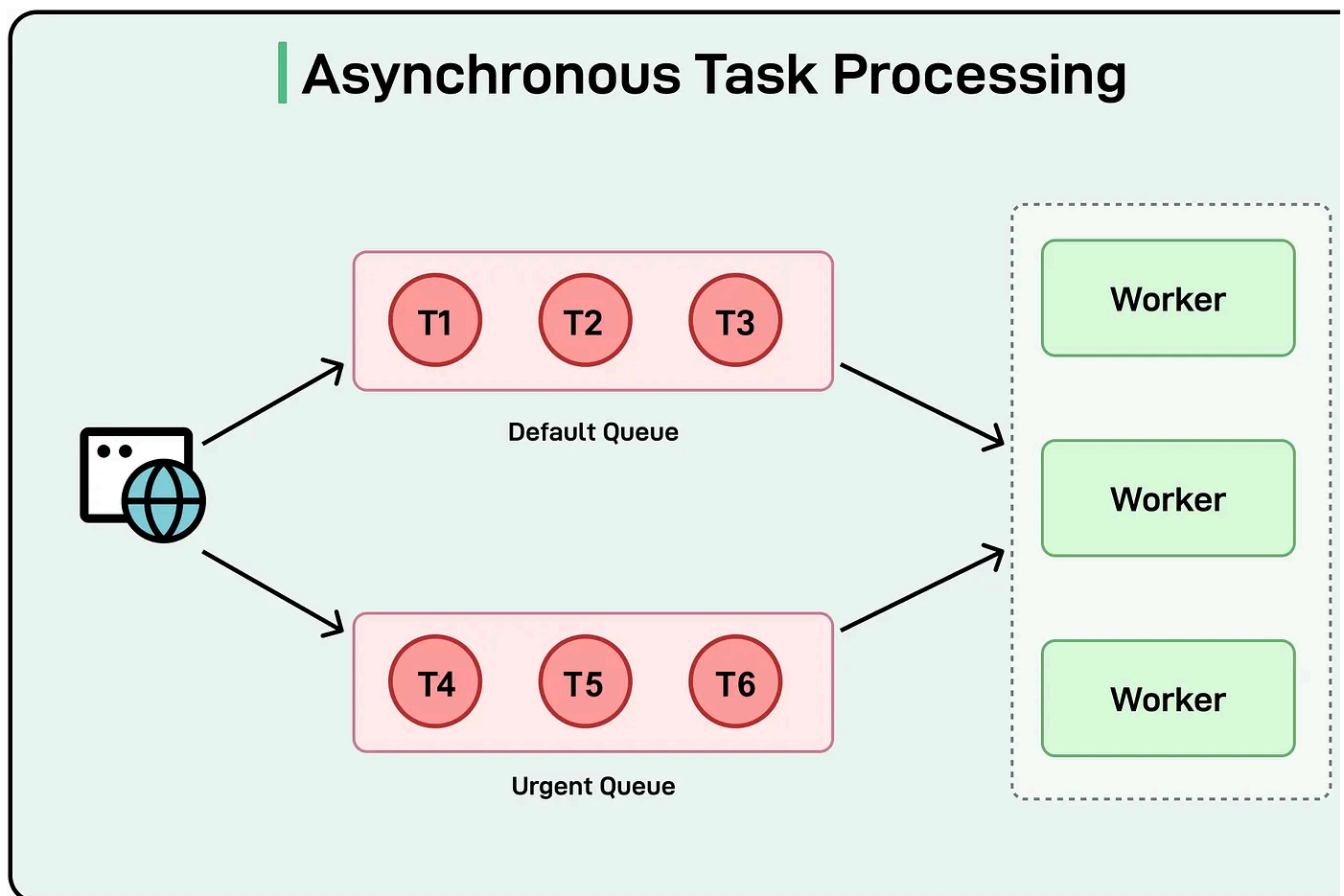
Asynchronous processing is an approach where tasks are executed independently and do not block the main thread or process.

Instead of waiting for a task to be completed, asynchronous systems proceed with other tasks and handle the results of the asynchronous operations when they are ready.

Asynchronous processing reduces latency in the following ways:

- **Avoiding Blocking Operations:** In synchronous systems, tasks are processed sequentially. If a task takes time (for example, waiting for a database response or an API call), subsequent tasks must wait. However, in asynchronous systems, tasks are initiated, and the system continues processing other tasks while waiting for the slower operation to complete.
- **Parallel Execution:** Asynchronous systems allow multiple tasks to run in parallel, reducing the time required to complete operations.
- **Improved Resource Utilization:** Resources such as CPU and memory are not idle while waiting for blocking tasks. Instead, they are utilized to process other requests or tasks.

See the diagram below that shows a simple task execution setup



There are different patterns to implement asynchronous processing:

- **Queue-based Systems:** Tasks are placed in a queue and processed independent often in the background. For example, a user submits a large file for processing. Instead of blocking the application, the request is added to a queue, and the user is notified when processing is complete.
- **Event-driven Architecture:** Applications are designed to react to events (for example, user actions or database changes) as they occur. For example, when a user clicks a button to send an email, the request triggers an event that processes the email asynchronously while the user continues interacting with the app.
- **Promises and Async/Await in Programming:** Promises and async/await syntax in programming languages like JavaScript handle asynchronous tasks gracefully. For example, fetching data from an API asynchronously in a web app to avoid freezing the user interface.

Some common use cases where asynchronous processing shines are as follows:

- Email notifications
- Payment processing
- Data processing in analytics workflows
- Chat applications

5 - Database Indexing

Database indexing is a technique that helps reduce the search time of a database query by using a special data structure known as the index.

An index is similar to an index in a book. It allows the database to quickly locate the data we are looking for without scanning the entire dataset.

See the diagram below for a simple example of an index based on the “email” field.

By optimizing data retrieval, indexes minimize the delay in responses, enhancing user experience and application performance. They also help reduce CPU and memory usage by avoiding full table scans, freeing resources for other operations.

Different types of indexes can be used:

- **Clustered Index:** A clustered index determines the physical order of data in a table. Each table can have only one clustered index. The data rows are stored directly in the order of the clustered index key. For example, a table with a clustered index on the EmployeeID column will physically store rows sorted by EmployeeID.
- **Non-Clustered Index:** A non-clustered index creates a separate structure that points to the location of the actual data in the table. It contains index keys and pointers (row locators) to the data rows. For example, an index on the LastName

column in an employee table allows quick lookups without altering the physical order of data. It is best for columns frequently used in search conditions, filters, or joins.

- **Composite Index:** This is an index on two or more columns. It is used to optimize queries that filter or sort by multiple columns. For example, an index on `FirstName` and `LastName` in a customer table speeds up searches for customers by their full name. It is useful for queries with multiple `WHERE` conditions or `ORDER BY` clauses.
- **Full-Text Index:** This is used for searching text data efficiently. It enables fast searches for text patterns or keywords within large text fields. For example, a full-text index on the `Description` column of a product catalog allows quick keyword searches. This is ideal for applications requiring advanced text search capabilities such as e-commerce or document management systems.

6 - Data Compression

Data compression is the process of reducing the size of data to minimize the time and bandwidth required for its transmission over a network. By decreasing the amount of data sent between the client and server, compression plays a vital role in reducing latency and improving application performance.

Compression is especially critical for resource-heavy web applications, video streaming services, and other bandwidth-intensive systems.

Here are the main ways data compression helps reduce latency:

- **Smaller Data Payloads:** Compression reduces the size of text, images, and other resources, enabling quicker transmission over the network.
- **Fewer Network Bottlenecks:** Lower data sizes decrease the strain on network infrastructure, improving overall performance during peak usage.

- **Improved Page Load Times:** Compressed web assets, like HTML, CSS, and JavaScript files, load faster in browsers, reducing perceived latency for users.

Some best practices for data compression are as follows:

- **Choose the Right Compression Method:** Use Brotli or Gzip for text assets depending on support by the client (browser). For images, use compression methods such as WebP or AVIF.
- **Balance Compression Quality and Performance:** Enable compression on serve (for example, NGINX, Apache) for specific file types like text/html, application/json, etc. Use lossy compression for non-critical images (such as thumbnails) and lossless for high-quality visuals.
- **Adaptive Compression:** Use different compression levels based on client bandwidth and device type. For example, serve highly compressed images to mobile users on slow connections while maintaining higher quality for desktop.
- **Pre-compress Static Assets:** Compress and store static assets (for example, HTML, CSS, JS files) during the build process to save server processing time.

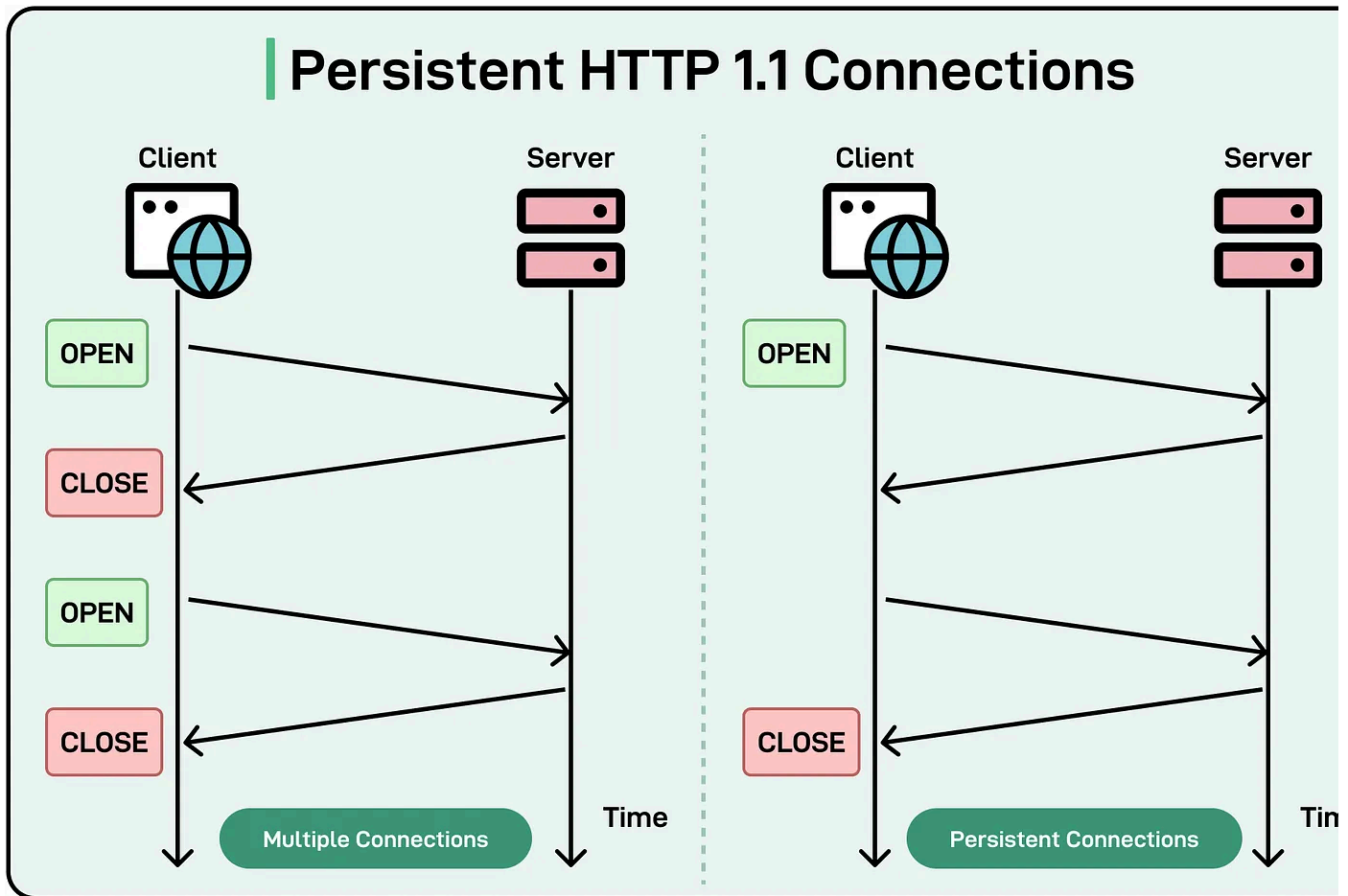
7 - Reusing Connections

Connection reuse, commonly implemented using the HTTP keep-alive feature, reduces latency by maintaining persistent connections between the client and server.

Establishing a new TCP connection involves a 3-way handshake (SYN, SYN-ACK, ACK), which takes time. For each new request, this handshake process introduces latency. This is where connection reuse becomes quite important.

For example, in HTTP/1.1, keep-alive is enabled by default, and connections remain open unless explicitly closed by including Connection: close in the request header. However, HTTP/1.1 handles only one request-response pair at a time per connection. Subsequent requests must wait for the previous ones to complete, a limitation known as head-of-line blocking.

See the diagram below that shows the difference between multiple connections and persistent connections in HTTP/1.1



HTTP/2 builds on the persistent connection concept by introducing multiplexing, allowing multiple requests and responses to be sent simultaneously over a single connection. Here's how multiplexing works:

- Requests and responses are broken into smaller frames, which are interleaved and sent over the same connection.
- The client and server can process multiple streams concurrently, eliminating head-of-line blocking.

See the diagram below that shows how multiplexing works in HTTP/2.

Some best practices for connection reuse are as follows:

- Ensure keep-alive is enabled in server configurations.
- Set reasonable timeouts for idle connections to balance resource usage and latency. For example, a timeout of 5-10 seconds suits most web applications.
- For modern applications, use HTTP/2 to take full advantage of multiplexed streams and faster resource loading.
- Use performance monitoring tools to track connection reuse and identify potential bottlenecks.

8 - Pre-caching

Pre-caching is the practice of proactively storing data or resources in a cache before the user explicitly requests them.

The idea is to cache data that is highly likely to be accessed in advance so that it can be served faster when a user requests it.

See the diagram below that shows one approach to implementing a pre-caching process:

By preparing data in advance, pre-caching reduces latency and ensures faster response times when the data is eventually needed.

Some examples of pre-caching in action are as follows:

- **Progressive Web Apps (PWAs):** PWAs use pre-caching to store essential files like HTML, CSS, JavaScript, and images during the initial installation phase. Service workers run in the background to intercept network requests and serve pre-cached resources.

- **Predictive Data Caching:** Applications predict the user's next actions and pre-fetch relevant data. For example, an e-commerce app may pre-fetch product details when the user hovers over or clicks a product category, ensuring the page loads instantly.
- **Video Pre-caching:** Video streaming apps buffer and pre-cache the next segment of a video while the current segment plays to support seamless playback.
- **Pre-fetching Content:** Websites pre-fetch content for links that users are likely click next. For example, a news website can pre-cache the content of trending articles displayed on the homepage.

Summary

In this article, we've taken a detailed look at the various strategies for reducing latency in applications.

Let's summarize our learnings in brief:

- Latency measures the time it takes for data to travel from a source to a destination and back.
- Latency can arise from multiple stages in data processing. There are different types of latencies such as network latency, server latency, and client-side latency.
- There are multiple strategies available to reduce latency. The choice of strategy depends on the project requirements.
- Caching frequently accessed data on the client, server, or edge reduces latency by minimizing trips to the database or origin server.
- CDNs cache content on geographically distributed servers, reducing latency by serving data from locations closer to users.
- Load Balancing allows the distribution of incoming traffic across multiple servers to prevent overload and ensure consistent response times.

- Asynchronous processing helps offload non-critical tasks to background processes to support faster response times.
- Use database indexing to speed up database queries and reduce server-side latency.
- Use data compression techniques to reduce the size of transmitted data. Tools Gzip or Brotli for text and WebP or AVIF for images can be used.
- Implement connection reuse by maintaining persistent connections between client and server to reduce the overhead of establishing new connections.
- Pre-cache essential assets and anticipated resources to ensure they load instantly when needed.

References:

- [Speed Matters by Google](#)
- [Amazon Found Every 100 ms latency cost them 1% in sales](#)



317 Likes • 15 Restacks

Discussion about this post

Comments Restacks



Write a comment...



José N'Sele Le Substack de José Jan 23, 2025

This technical description about latency is the most complete, the clearest and the best graphical explanation I ever read on this complex subject. I've really appreciate it. Thank You so much!

♡ LIKE (2) 💬 REPLY



Dmitry Jan 23, 2025

Thank you for the great article!

♡ LIKE 💬 REPLY



© 2026 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture